

Apply Filters to SQL Queries

Maximilian Rivera | Security Portfolio | June 2026

PROJECT DESCRIPTION

This activity demonstrates the use of SQL filters to investigate potential security incidents within an organization's database. Using the `log_in_attempts` and `employees` tables, I applied a series of targeted queries to identify failed login activity, suspicious date-based patterns, and employee machine assignments across specific departments and office locations.

Each query was built using SQL operators and clauses — including `AND`, `OR`, `NOT`, `LIKE`, and comparison operators — to isolate records that matched investigative criteria. The goal was to support a security team in determining the scope of two separate events: unusual after-hours login behavior and unauthorized access attempts, and in preparing for targeted machine security updates across multiple departments.

RETRIEVE AFTER HOURS FAILED LOGIN ATTEMPTS

A potential security incident occurred after business hours. I needed to identify all failed login attempts that took place after 18:00 to determine whether unauthorized access was attempted outside of normal working hours.

Query:

```
SELECT *  
FROM log_in_attempts  
WHERE login_time > '18:00' AND success = 0;
```

This query retrieves all columns from the `log_in_attempts` table. The `WHERE` clause uses two conditions joined by `AND`: `login_time > '18:00'` filters for attempts that occurred after 6:00 PM, and `success = 0` filters specifically for failed attempts (a value of 0 or `FALSE` indicates failure). Both conditions must be true for a record to appear in the results. This narrowed the dataset to only the after-hours failed logins relevant to the investigation.

RETRIEVE LOGIN ATTEMPTS ON SPECIFIC DATES

A suspicious event was flagged on 2022-05-09. To get a complete picture of that window, I queried for all login attempts on that date and the day before — 2022-05-08 — to identify any related activity leading up to the incident.

Query:

```
SELECT *  
FROM log_in_attempts  
WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

This query returns all records from `log_in_attempts` where the `login_date` matches either 2022-05-09 or 2022-05-08. The OR operator ensures that records matching either date are included — unlike AND, which would require both conditions to be true simultaneously, which is impossible for a single date value. This returned the full login activity for both days for review.

RETRIEVE LOGIN ATTEMPTS OUTSIDE OF MEXICO

After reviewing the suspicious login activity, the team determined it did not originate from Mexico. I needed to filter out all Mexican login attempts from the dataset, accounting for the fact that the `country` column stores Mexico as both 'MEX' and 'MEXICO'.

Query:

```
SELECT *  
FROM log_in_attempts  
WHERE NOT country LIKE 'MEX%';
```

The NOT operator excludes any records matching the specified condition. LIKE 'MEX%' uses the wildcard character % to match any string starting with 'MEX' — this covers both 'MEX' and 'MEXICO' in a single pattern. Combining NOT with LIKE ensures all Mexican entries are excluded regardless of which abbreviation was used, and the results reflect only login attempts originating outside Mexico.

RETRIEVE EMPLOYEES IN MARKETING

The security team needed to push updates to machines belonging to Marketing department employees specifically located in the East building. Office values follow a format like 'East-170' or 'East-320', so I used pattern matching to capture all East offices.

Query:

```
SELECT *  
FROM employees  
WHERE department = 'Marketing' AND office LIKE 'East%';
```

The WHERE clause requires both conditions to be true: `department = 'Marketing'` matches only Marketing employees, and `office LIKE 'East%'` uses the % wildcard to match any office value beginning with 'East'. The AND operator ensures only records satisfying both criteria are returned — employees in Marketing who are also located in an East building office. This gave the team the exact list of machines to target.

RETRIEVE EMPLOYEES IN FINANCE OR SALES

A separate security update needed to be applied to machines belonging to employees in either the Finance or Sales departments. I queried the `employees` table to return all records matching either department.

Query:

```
SELECT *  
FROM employees  
WHERE department = 'Finance' OR department = 'Sales';
```

The OR operator returns records where at least one of the conditions is true. Since a single employee can only belong to one department, using OR here captures all employees in Finance as well as all employees in Sales in a single result set. This is the correct operator when the target records can satisfy either condition independently.

RETRIEVE ALL EMPLOYEES NOT IN IT

The final update had already been applied to Information Technology department machines. I needed to identify all remaining employees — every department except IT — so the update could be pushed to their devices.

Query:

```
SELECT *  
FROM employees  
WHERE NOT department = 'Information Technology';
```

The NOT operator negates the condition, returning all records where the department is not 'Information Technology'. This is more efficient than listing every other department individually and ensures no department is accidentally excluded. The result set contains every employee outside of IT, giving the team the complete list of machines that still need the update applied.

SUMMARY

Across six queries, I used SQL filters to support two parallel security investigations: one focused on suspicious login activity and one focused on preparing machines for security updates. For the login analysis, I applied AND to isolate after-hours failures, OR to capture a two-day date window, and NOT with LIKE to exclude Mexican logins while handling two different country code formats in the same column.

For the employee machine updates, I used AND with LIKE to target a specific department and building, OR to pull employees from two different departments, and NOT to exclude the one department that had already been updated. These queries demonstrate how SQL's logical operators — AND, OR, and NOT — combined with pattern matching through LIKE enable precise, efficient data retrieval in real security investigations.